Smart Backup Kernel-Space Utility

Informe Técnico de Análisis y Resultados

Materia	Sistemas Operativos – ST0263 / SI3007
Universidad	EAFIT – Ingeniería de Sistemas
Integrantes	Juan José Barón Estefanía Ramírez Juan Camilo Gómez Jiménez
Plataforma	Linux x86-64 · WSL2 / Ubuntu · GCC -std=c11 -O2
Fecha	2025

1. Introducción

El presente trabajo documenta el diseño, implementación y análisis de rendimiento del **Smart Backup Kernel-Space Utility**, un sistema de respaldo de archivos escrito en C que opera en dos capas diferenciadas. La primera, *sys_smart_copy*, simula el comportamiento de una syscall del kernel usando descriptores de archivo POSIX de bajo nivel. La segunda, *stdio_copy*, emplea la biblioteca estándar stdio.h como punto de comparación. El objetivo central es entender el impacto de los *context switches* en el rendimiento de operaciones de I/O, tanto en condiciones normales como en entornos virtualizados como WSL2.

2. Diseño de sys_smart_copy

2.1 Por qué se considera una función de sistema

Una syscall opera en modo kernel (ring 0), con acceso directo al VFS y al subsistema de bloques. *sys_smart_copy* replica este modelo desde espacio de usuario porque usa **open(2)**, **read(2)**, **write(2)** y **close(2)** directamente sin ninguna capa de abstracción adicional. El buffer se dimensiona en **PAGE_SIZE = 4096 bytes**, la misma unidad que usa el kernel para gestionar memoria virtual en x86-64. La actividad se registra mediante **syslog(3)**, equivalente funcional de *printk(KERN_INFO)* en un módulo real. El manejo de escrituras parciales se hace con un bucle interno, exactamente como lo haría la capa de bloque del kernel ante I/O fragmentado.

2.2 Firma y flags de control

La función se declara en **smart_copy.h** con la siguiente firma:

```
int sys_smart_copy(const char *src_path, const char *dst_path,
                   uint32_t flags, sc_stats_t *stats);
```

Flag	Hex	Descripción
`SC_FLAG_NONE`	`0x00`	Copia plana, sin opciones adicionales

`SC_FLAG_CHECK_PERMS`	`0x01`	Verifica permisos con access(2) antes de abrir
`SC_FLAG_LOG`	`0x02`	Registra en syslog y stderr (equiv. printk)
`SC_FLAG_LARGE_BUF`	`0x04`	Buffer de 1 MiB para archivos grandes
`SC_FLAG_SYNC`	`0x08`	fsync() al cerrar — garantía de persistencia

Tabla 1. Flags de control de sys_smart_copy.

2.3 Gestión de errores con errno

Cada syscall crítica se verifica. El patrón **goto cleanup** centraliza el cierre de descriptores y la liberación del buffer, garantizando que no queden recursos abiertos independientemente de la ruta de error:

errno	Causa	Punto de detección
`ENOENT`	Archivo origen no existe	open(2) origen
`EACCES`	Permisos insuficientes	access(2) / open(2)
`ENOSPC`	Disco lleno durante escritura	write(2) retorna < 0
`ENOMEM`	Fallo al asignar el buffer	malloc() retorna NULL
`EIO`	Error de I/O en lectura	read(2) retorna < 0
`EINVAL`	Puntero NULL recibido	Validación de entrada

Tabla 2. Errores manejados y su origen.

3. Comparativa: Syscall directa vs stdio

3.1 El context switch como costo fundamental

Cuando un proceso invoca una syscall, el procesador transiciona de modo usuario (ring 3) a modo kernel (ring 0). Este cambio implica guardar el estado completo del proceso, cambiar el espacio de direcciones virtuales y ejecutar el handler en modo privilegiado. En hardware nativo, cada context switch cuesta entre **1 y 10 microsegundos**. En WSL2, la capa de virtualización Hyper-V puede multiplicar este costo entre 2 y 5 veces.

3.2 Cómo stdio reduce los context switches

sys_smart_copy emite una syscall read(2) y una syscall write(2) por cada bloque de 4 KB. Para un archivo de 1 MB esto genera **512 context switches** (256 read + 256 write). Para 1 GB genera **524.288 context switches**.

stdio_copy mantiene un buffer interno en espacio de usuario de ~8 KB (glibc). Los datos se acumulan ahí y la syscall real se emite solo cuando el buffer se llena. Para archivos menores a 8 KB es posible completar la copia sin emitir ninguna syscall adicional, reduciendo el overhead a casi cero.

3.3 Comportamiento específico en WSL2

En WSL2 el filesystem nativo de Linux (/tmp/) tiene buen rendimiento, pero el costo amplificado del context switch hace que la ventaja del buffer de stdio sea más pronunciada que en bare metal. Adicionalmente, WSL2 comparte los recursos de CPU con Windows, lo que introduce variabilidad en los tiempos medidos, especialmente visible en las pruebas de 1 GB.

4. Resultados del Benchmark

Configuración: 5 repeticiones por método y tamaño, medición con CLOCK_MONOTONIC, archivos generados con datos pseudo-aleatorios en /tmp/ (filesystem nativo de WSL2). Los tiempos incluyen apertura y cierre de descriptores.

Tamaño	Método	Min (ms)	Max (ms)	Avg (ms)	Bloques
1 KB	sys_smart_copy	0.014	0.134	0.062	1
1 KB	**stdio_copy**	**0.015**	**0.062**	**0.032**	**—**
1 MB	sys_smart_copy	0.748	1.573	1.076	256
1 MB	**stdio_copy**	**0.651**	**0.742**	**0.677**	**—**
1 GB	sys_smart_copy	564.900	1074.609	741.699	262.144
1 GB	**stdio_copy**	**566.659**	**758.718**	**674.105**	**—**

Tabla 3. Tiempos de ejecución medidos en WSL2/Ubuntu. Verde: método más eficiente por tamaño.

```
╔═══════════════════════════════════════════════════════════════════╗
║         SMART BACKUP  -  Benchmark: sys_smart_copy vs stdio_copy  ╒
║         Plataforma: Linux (POSIX)   Runs por método: 5            ╒
╚═══════════════════════════════════════════════════════════════════╝

Nota: CLOCK_MONOTONIC elimina distorsiones por cambios de hora del sistema.
      Los tiempos incluyen syscalls de apertura/cierre de descriptores.

+----------------+------------+-----------+------------+-----------+-----------+
| Tamaño         | Método     | Min (ms)  | Max (ms)   | Avg (ms)  | Bloques   |
+----------------+------------+-----------+------------+-----------+-----------+
| Generando 1 KB...                                                             |
| 1 KB           | sys_smart  | 0.014     | 0.134      | 0.062     | 1         |
| 1 KB           | stdio      | 0.015     | 0.062      | 0.032     | N/A       |
+----------------+------------+-----------+------------+-----------+-----------+
| Generando 1 MB...                                                             |
| 1 MB           | sys_smart  | 0.748     | 1.573      | 1.076     | 256       |
| 1 MB           | stdio      | 0.651     | 0.742      | 0.677     | N/A       |
+----------------+------------+-----------+------------+-----------+-----------+
| Generando 1 GB...                                                             |
| 1 GB           | sys_smart  | 564.900   | 1074.609   | 741.699   | 262144    |
| 1 GB           | stdio      | 566.659   | 758.718    | 674.105   | N/A       |
+----------------+------------+-----------+------------+-----------+-----------+

╔═══════════════════════════════════════════════════════════════════╗
║  ANÁLISIS: ¿Por qué stdio es más eficiente en archivos pequeños?  ╒
╚═══════════════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════════════╗
║                                                                   ╒
║  sys_smart_copy usa read(2)/write(2) directamente:                ╒
║    → Cada llamada provoca un CONTEXT SWITCH user-space → kernel.  ╒
║    → Para 1 KB con buf=4KB: 1 read + 1 write = 2 context switches. ╒
║    → El overhead de cada context switch (~1-10 µs) domina el tiempo. ╒
║                                                                   ╒
║  stdio_copy usa fread()/fwrite() con buffer interno (glibc, ~8 KB): ╒
║    → Para archivos < 8 KB es posible completar la copia sin emitir ╒
║      ninguna syscall adicional (datos quedan en buffer de usuario). ╒
║    → Menos context switches = menor latencia para archivos pequeños. ╒
║                                                                   ╒
║  Para archivos grandes (≥ 1 MB):                                  ╒
║    → Ambos métodos emiten múltiples syscalls y el overhead se diluye. ╒
║    → sys_smart_copy con SC_FLAG_LARGE_BUF (1 MiB) puede superar a  ╒
║      stdio al reducir el número total de context switches.        ╒
╚═══════════════════════════════════════════════════════════════════╝

jcamilogj@Camilo:~/smart-backup-so$
L: Ubuntu   ⊗ 0 ⚠ 0   0   Update is ready, click to restart.
```

Figura 1. Captura de terminal — ejecución real del benchmark en WSL2/Ubuntu.

4.1 Análisis por tamaño

1 KB: stdio_copy (0.032 ms avg) supera a sys_smart_copy (0.062 ms avg) por un margen del 48%. Aunque ambos usan un solo bloque de 4 KB, el overhead de inicialización de los descriptores POSIX de bajo nivel es más costoso que la apertura de un FILE* en este contexto virtualizado.

1 MB: stdio_copy (0.677 ms avg) supera a sys_smart_copy (1.076 ms avg) por un margen del 37%. A 256 bloques, sys_smart_copy acumula 512 context switches mientras stdio los amortigua con su buffer

interno. La variabilidad de sys_smart_copy (0.748–1.573 ms) refleja sensibilidad al scheduler de WSL2.

1 GB: stdio_copy (674 ms avg) supera a sys_smart_copy (741 ms avg) por un margen del 9%. Con 262.144 bloques de 4 KB, sys_smart_copy acumula más de 524.000 context switches. La variabilidad de sys_smart_copy (564–1074 ms) sigue siendo mayor que la de stdio (566–758 ms), confirmando que más context switches implica mayor exposición a interrupciones del scheduler.

Conclusión: stdio_copy es consistentemente más eficiente en todos los tamaños probados bajo WSL2. La activación de **SC_FLAG_LARGE_BUF** en sys_smart_copy reduciría los bloques de 1 GB de 262.144 a 1.024, lo que potencialmente eliminaría esta diferencia en Linux nativo.

5. Pruebas Funcionales

5.1 Copia exitosa con flags de permisos y log (flags=0x03)

Comandos ejecutados en terminal Ubuntu (WSL2):

```
$ echo "Hola EAFIT Sistemas Operativos" > origen.txt
$ ./backup origen.txt destino.txt 3
```

Salida obtenida:

```
[backup] Copiando 'origen.txt' → 'destino.txt' (flags=0x03)
[smart_copy] Iniciando copia: 'origen.txt' → 'destino.txt' flags=0x03
[smart_copy] Copia completada: 31 bytes en 1 bloque(s). buf=4096 B
[backup] ✓ Éxito: 31 bytes copiados en 1 bloques (buf=4096 B)
$ diff origen.txt destino.txt && echo 'Archivos identicos'
Archivos identicos
```

SC_FLAG_CHECK_PERMS (0x01) verificó permisos con access(2) antes de abrir los descriptores. SC_FLAG_LOG (0x02) registró inicio y fin en syslog y stderr. El diff confirma integridad binaria de la copia.

5.2 Manejo de error: archivo inexistente (ENOENT)

Comando ejecutado:

```
$ ./backup noexiste.txt destino.txt 2
```

Salida obtenida:

```
[backup] Copiando 'noexiste.txt' → 'destino.txt' (flags=0x02)
[smart_copy] Iniciando copia: 'noexiste.txt' → 'destino.txt' flags=0x02
[smart_copy ERROR] open() origen: No such file or directory
[backup] ✗ Error al copiar el archivo.
```

El sistema detectó ENOENT en open(2), reportó el error con contexto operacional, cerró todos los recursos vía goto cleanup y retornó EXIT_FAILURE al proceso padre. No quedaron descriptores abiertos ni memoria sin liberar.

6. Conclusiones

1. Fidelidad al modelo de syscall: sys_smart_copy replica los aspectos esenciales de una función de sistema al operar sobre primitivas POSIX de bajo nivel, usar buffers alineados a PAGE_SIZE y registrar actividad en syslog. La diferencia con un módulo de kernel real es únicamente el nivel de privilegio (ring 3 vs ring 0) y el uso de malloc en lugar de kmalloc.

2. El context switch como variable crítica: Los resultados demuestran que stdio_copy supera a sys_smart_copy en todos los tamaños probados bajo WSL2. El buffer interno de glibc reduce la frecuencia de context switches, y en un entorno virtualizado donde ese costo está amplificado, la ventaja es consistente desde 1 KB hasta 1 GB.

3. Variabilidad como indicador de diseño: sys_smart_copy exhibe mayor variabilidad en todos los tamaños, especialmente en 1 GB (564–1074 ms vs 566–758 ms de stdio). Más context switches implica mayor exposición a interrupciones del scheduler — un factor crítico en sistemas de producción.

4. Robustez: El patrón goto cleanup garantiza liberación de recursos en toda ruta de error. Las pruebas funcionales confirmaron manejo correcto de ENOENT, registro en syslog y verificación de integridad mediante diff.

Juan José Barón · Estefanía Ramírez · Juan Camilo Gómez Jiménez
Sistemas Operativos · Universidad EAFIT · 2025